04

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

이벤트 루프와 논블로킹 I/O

제1부 기반: 백엔드와 Node.js 런타임 · 2장 이벤트 루프와 비동기 I/O

스레드 하나로 요청 수천 개를 받는 방법, 한꺼번에 멈추는 이유



© 한상훈 NestJS 백엔드 마스터

직원 한 명뿐인 카페

→ 주문 → 에스프레소 머신에 샷 걸기 → 다음 손님 주문

→ 샷 완성 → 컵에 담아 전달

→ 직원 한 명, 줄은 계속 줄어듦



© 한상훈 NestJS 백엔드 마스터

원두를 손으로 갈기 시작하면

→ 손이 묶인 동안: 주문 못 받음, 완성된 샷도 못 냄

→ 줄 전체 정지



© 한상훈 NestJS 백엔드 마스터

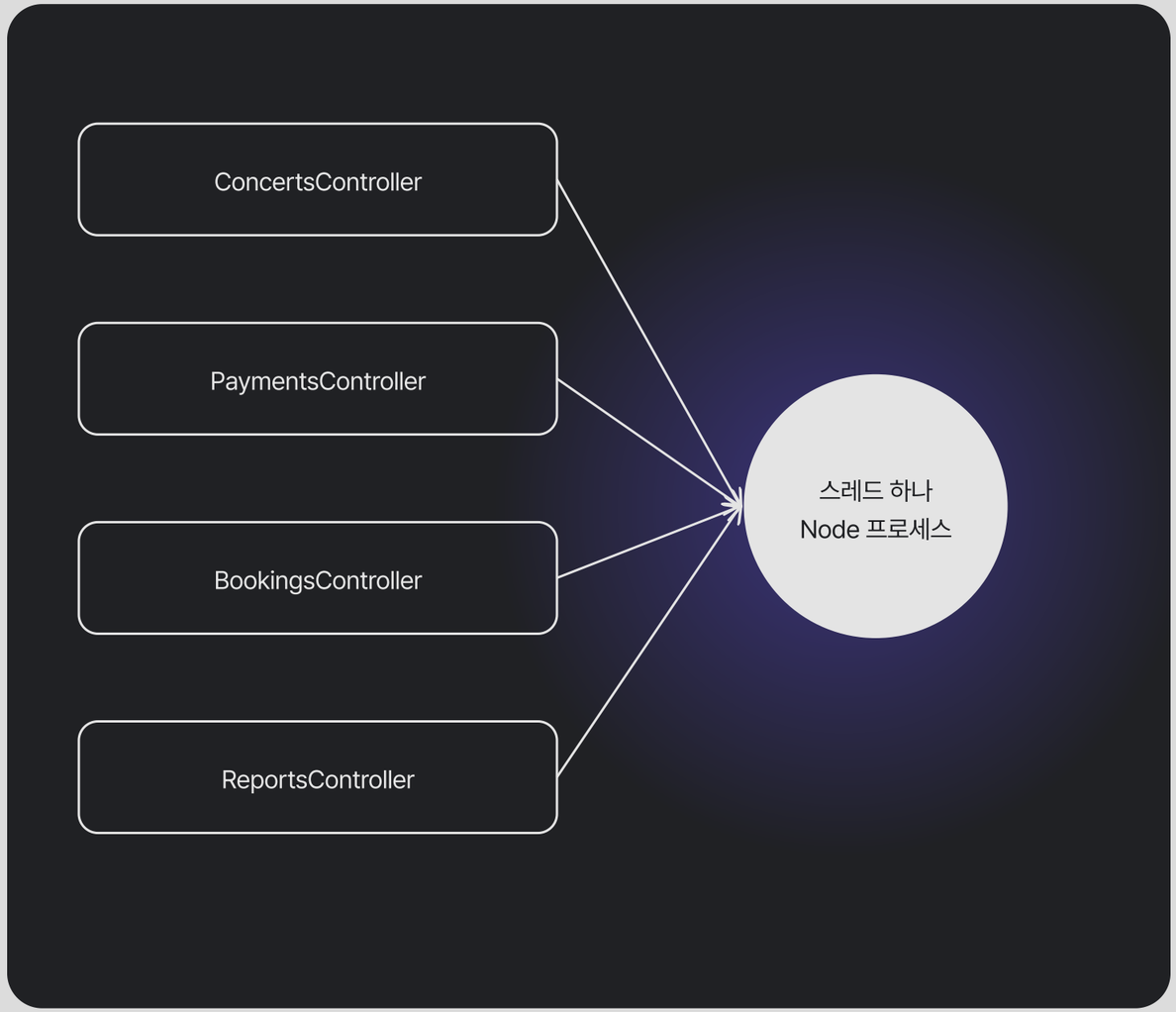
컨트롤러가 백 개여도 스레드는 하나

→ NestJS 서버 = Node 프로세스 하나

→ 자바스크립트 실행 스레드: 하나

→ 컨트롤러 하나의 코드 한 줄 → 다른 컨트롤러의 손님까지 대기

→ 비교: 톰캣 같은 서버는 요청마다 스레드 하나 (예: 스레드 200개 → 200명 동시)



© 한상훈 NestJS 백엔드 마스터

엔드포인트 두 개

server.mjs

```mjs
import http from 'node:http';

function busy(ms) {
  const end = Date.now() + ms;
  while (Date.now() < end) {}
}

http.createServer((req, res) => {
  if (req.url === '/wait') {
    // DB 응답을 1초 기다리는 흉내
    setTimeout(() => res.end('wait done\n'), 1000);
  } else if (req.url === '/block') {
    // 1초 동안 CPU로 계산하는 흉내
    busy(1000);
    res.end('block done\n');
  }
}).listen(4004);
```

→ 둘 다 응답까지 1초

© 한상훈 NestJS 백엔드 마스터

/wait 동시 요청 10개

터미널

```
$ node client.mjs /wait 10
#2 done 1060ms
#1 done 1071ms
#3 done 1075ms
...
#8 done 1089ms
#10 done 1090ms
10 requests to /wait: 1090ms
```

→ 열 개 모두 약 1초에 완료

© 한상훈 NestJS 백엔드 마스터

/block 동시 요청 10개

터미널

```
$ node client.mjs /block 10
#2 done 1040ms
#1 done 2035ms
#4 done 3035ms
#3 done 4036ms
...
#8 done 9036ms
#10 done 10036ms
10 requests to /block: 10036ms
```

→ 1초 간격으로 하나씩 완료

→ 동시 1,000명이면: /wait는 1초 남짓, /block은 마지막 손님 1,000초

© 한상훈 NestJS 백엔드 마스터

1초와 10초

엔드포인트	1초 동안 스레드	열 명 전체	마지막 손님
/wait	빈 상태, 다른 요청 처리	1,090ms	1초
/block	계산 중, 다른 요청 대기	10,036ms	10초

데이터 블록: 2026-10-26 강사 PC 실측, Node v24.11.0 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

블로킹 I/O와 논블로킹 I/O

구분	스레드가 하는 일	결과를 받는 법
블로킹 I/O	결과가 올 때까지 그 자리에서 대기	함수의 반환값
논블로킹 I/O	일을 맡기고 바로 다음 코드로	나중에 콜백, 프로미스

→ I/O: 디스크, 네트워크, DB처럼 프로세스 밖과 주고받는 일

© 한상훈 NestJS 백엔드 마스터

readFileSync와 readFile

```
import { readFileSync } from 'node:fs';
import { readFile } from 'node:fs/promises';

// 블로킹: 파일을 다 읽을 때까지 스레드가 여기서 멈춤
const a = readFileSync('seats.json');

// 논블로킹: 읽기를 맡겨 두고 스레드는 다른 요청으로
const b = await readFile('seats.json');
```

→ `await`: 이 함수만 다음 줄을 나중으로 미룸, 스레드는 자유

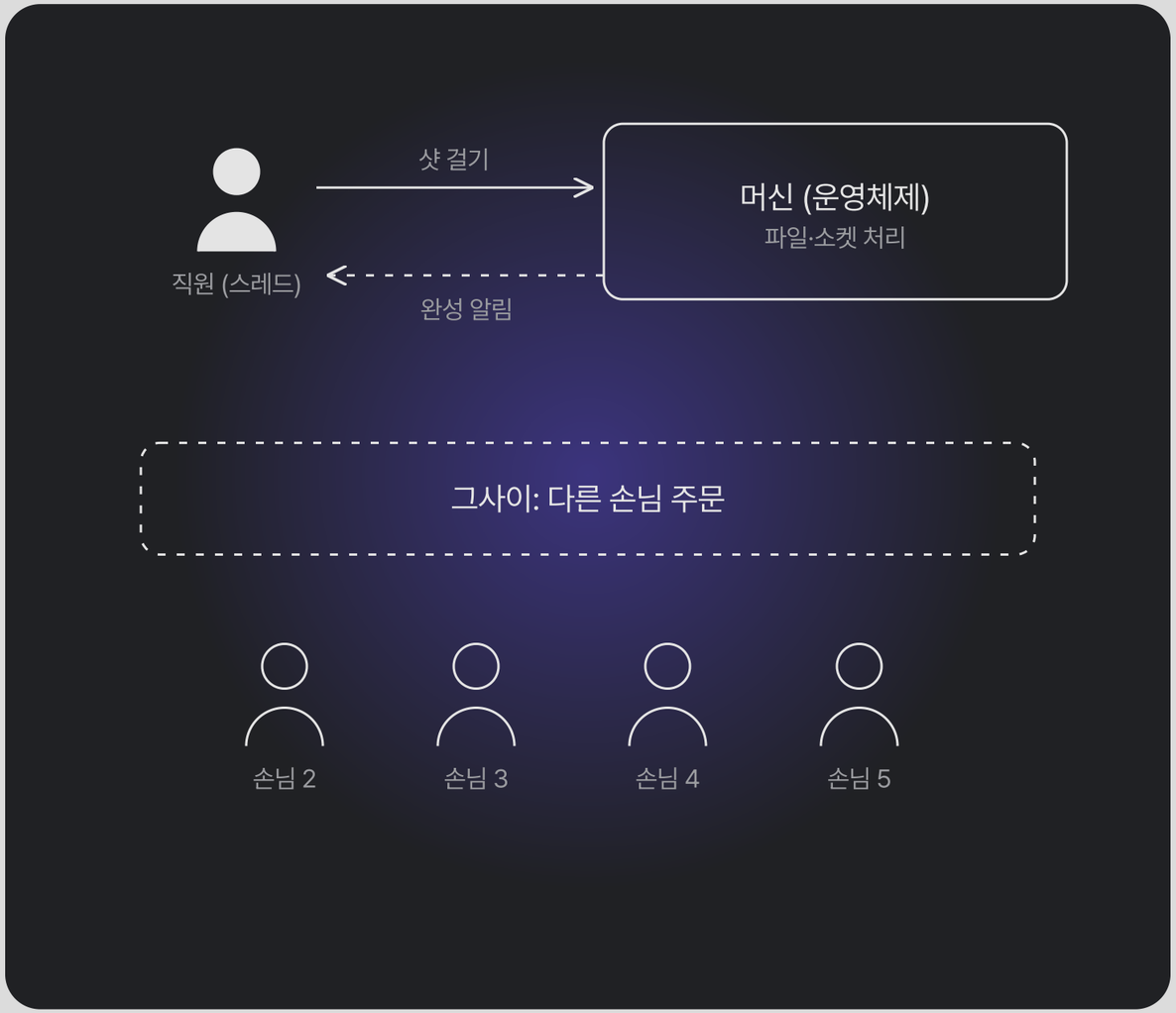
© 한상훈 NestJS 백엔드 마스터

샷을 걸어 두는 직원

→ 샷 걸기 = 읽기 요청을 맡김

→ 머신이 도는 동안 = 운영체제가 파일·소켓 처리

→ 샷 완성 알림 = 결과 도착, 다음 코드 실행



© 한상훈 NestJS 백엔드 마스터

Sync가 붙은 함수

```
fs.readFileSync
fs.writeFileSync
fs.existsSync
crypto.pbkdf2Sync
crypto.scryptSync
zlib.gzipSync
```

→ 서버 시작 때 한 번: 문제없음

→ 예: 좌석 배치도 JSON을 요청마다 `readFileSync` → 오픈 순간 수만 번 차례로 대기

→ 고치는 법: 시작 때 한 번 읽어 메모리에, 또는 `await readFile`

→ 요청을 처리하는 코드 안: 다른 요청 전부 대기

© 한상훈 NestJS 백엔드 마스터

이벤트 루프

→ 할 일이 남아 있는 동안 계속 도는 반복문

→ 한 바퀴: 끝난 일 확인 → 그 콜백 실행 → 다음 바퀴

→ 할 일: 걸어 둔 타이머, 열린 소켓, 맡겨 둔 I/O

→ 할 일이 하나도 없으면 프로세스 종료

© 한상훈 NestJS 백엔드 마스터

한 바퀴의 단계

→ `timers`: 시간이 된 setTimeout, setInterval

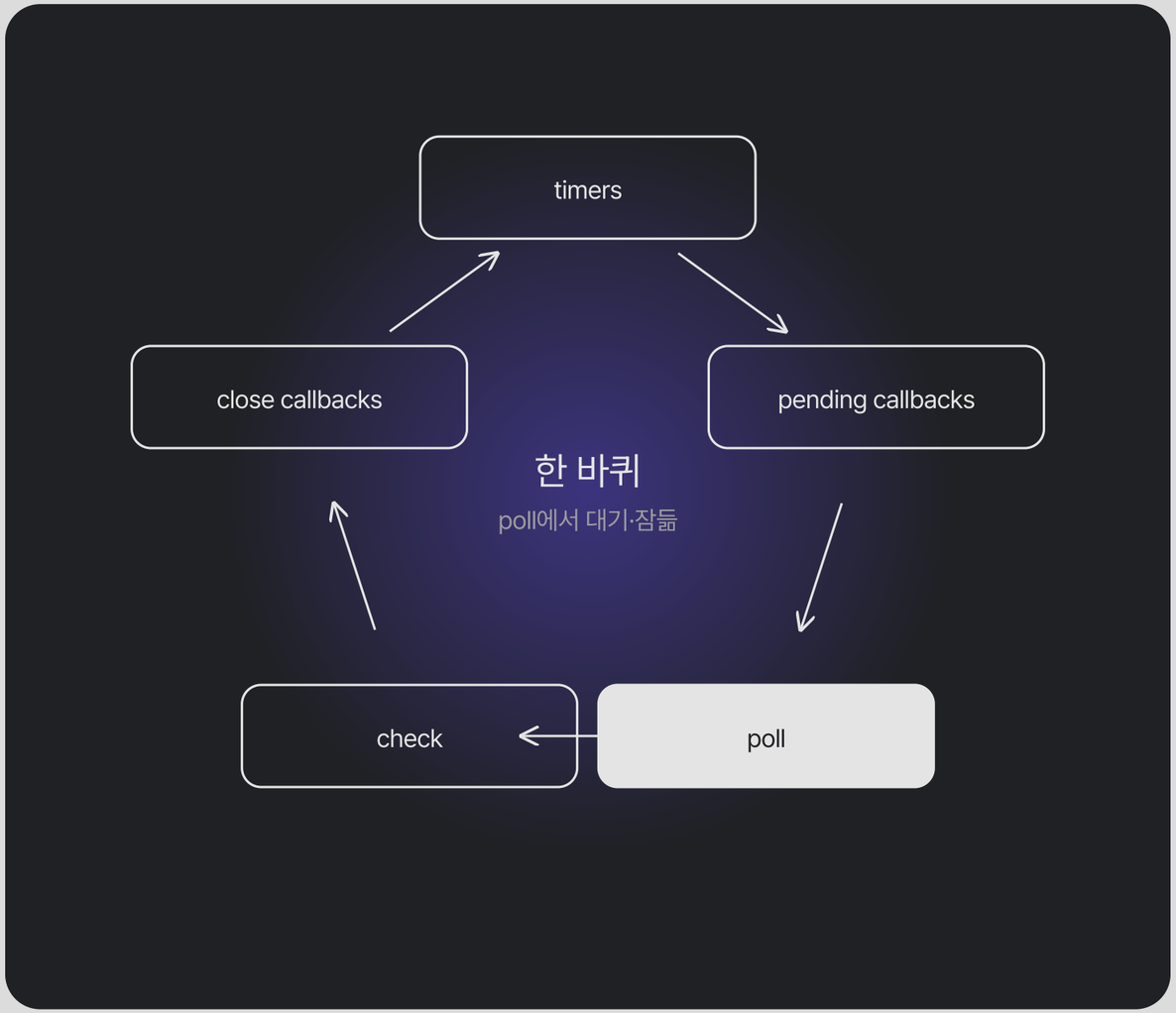
→ `pending callbacks`: 미뤄진 일부 I/O 콜백

→ `poll`: 끝난 I/O 확인, 그 콜백 실행, 할 일 없으면 여기서 대기

→ `check`: setImmediate

→ `close callbacks`: 닫힌 소켓의 close 이벤트

→ 할 일이 없을 때: poll 칸에서 운영체제의 알림을 기다리며 잠듦, CPU 거의 0



© 한상훈 NestJS 백엔드 마스터

I/O 콜백 안의 setTimeout과 setImmediate

phases.mjs

```
import { readFile } from 'node:fs';

readFile(import.meta.filename, () => {
  setTimeout(() => console.log('setTimeout 0'), 0);
  setImmediate(() => console.log('setImmediate'));
});
```

○ ○ ○ 터미널

```
$ node phases.mjs
setImmediate
setTimeout 0
```

→ 세 번 실행, 세 번 모두 같은 순서

© 한상훈 NestJS 백엔드 마스터

루프가 일을 꺼내는 때

순서

콜백 하나가 끝나야 다음 콜백

실행 중

콜백 실행 중: 다른 칸으로 못 감

1초

콜백 하나가 1초 = 루프 한 바퀴가 1초 이상

© 한상훈 NestJS 백엔드 마스터

정산 리포트와 헬스 체크

server.mjs

```
} else if (req.url === '/health') {
  res.end('ok\n');
} else if (req.url === '/report') {
  busy(2000);   // 2초짜리 정산 계산 흉내
  res.end('report done\n');
}
```

→ /health 혼자: 즉시 응답

© 한상훈 NestJS 백엔드 마스터

1.9초 걸린 /health

터미널

```
$ node health.mjs
/health  sent at 135ms, took 1901ms
/report  done at 2035ms

$ node health.mjs
/health  sent at 139ms, took 1910ms
/report  done at 2047ms
```

→ /report 호출 0.1초 뒤 /health 호출

→ /health 응답: /report 계산이 끝난 뒤

© 한상훈 NestJS 백엔드 마스터

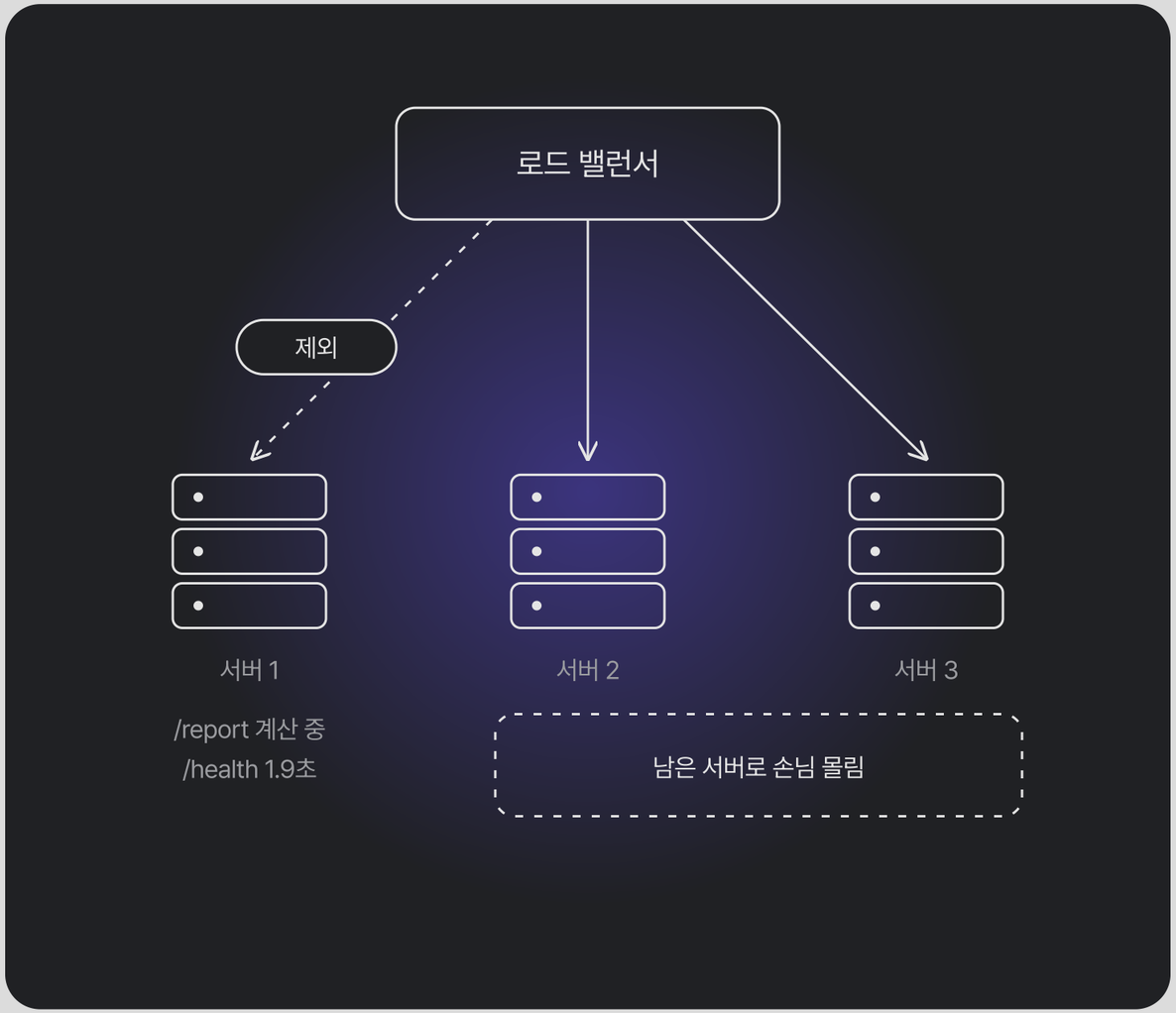
헬스 체크 대기 1초인 로드 밸런서

→ 헬스 체크: 로드 밸런서가 서버마다 주기적으로 보내는 "살아 있나" 요청

→ 대기 1초 안에 답이 없으면 장애로 판단, 손님 분배에서 제외

→ 정산 요청 하나 → 멀쩡한 서버가 목록에서 빠짐

→ 남은 서버로 손님 몰림 → 그 서버에도 정산 요청이 오면 연쇄



© 한상훈 NestJS 백엔드 마스터

컨트롤러 둘, 루프 하나

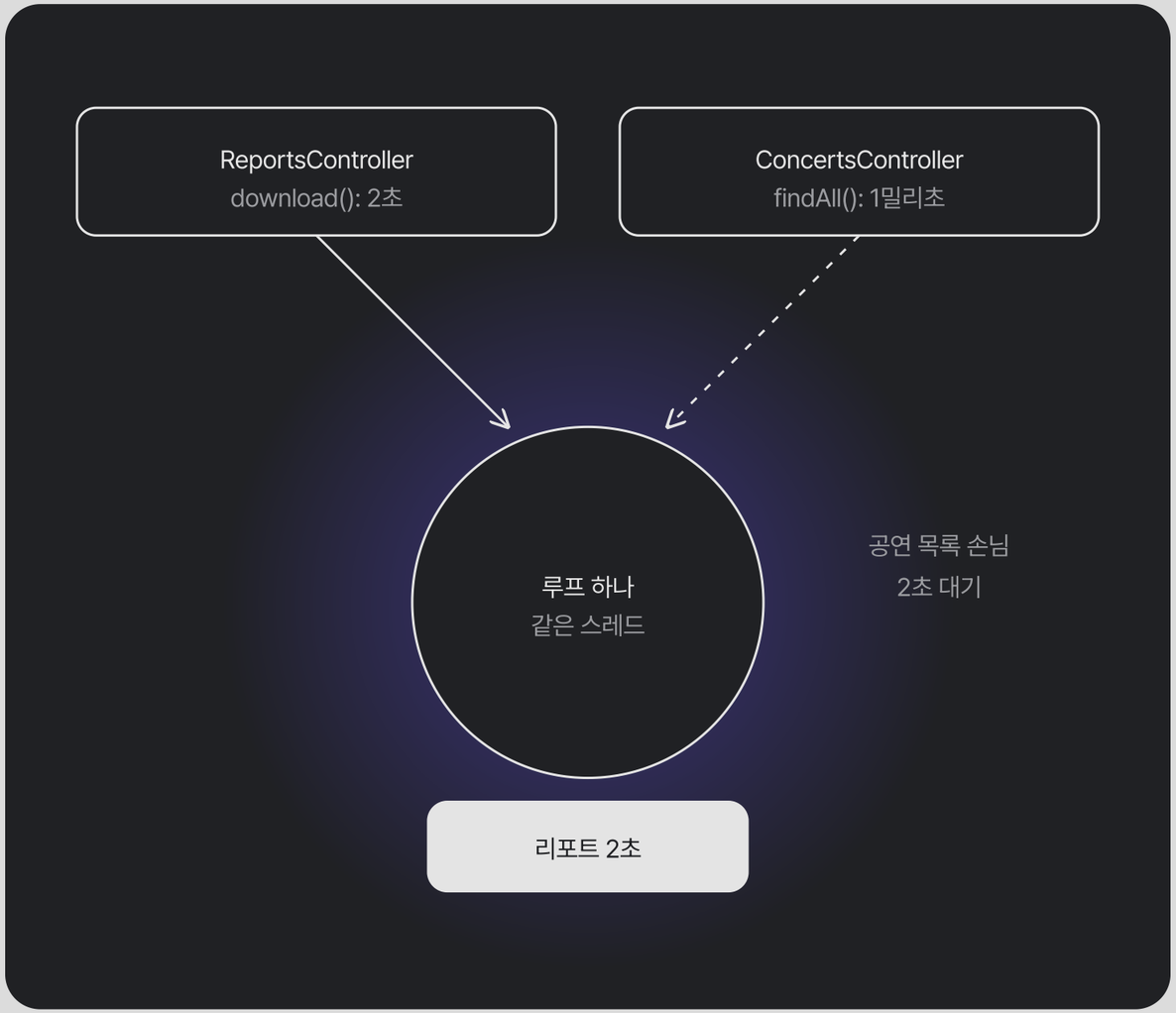
→ `ReportsController.download()`: 2초 계산

→ `ConcertsController.findAll()`: 1밀리초 조회

→ 같은 프로세스, 같은 스레드

→ 공연 목록 손님도 2초 대기

→ DB 쿼리 시간은 정상: 결과를 받아 줄 스레드가 다른 일 중



© 한상훈 NestJS 백엔드 마스터

요청 경로의 동기 코드

코드	입력	걸린 시간
`JSON.stringify`	예매 내역 20만 건 (18.3MB)	52ms
`JSON.parse`	같은 문자열	80ms
`pbkdf2Sync`	반복 60만 회	76ms
`readFileSync`	같은 18.3MB 파일	5ms

→ 그동안 다른 요청 전부 대기

데이터 블록: 2026-10-26 강사 PC 실측, Node v24.11.0 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

예매 내역 20만 건 JSON

→ 관리자 화면의 "전체 내역 내려받기"

→ 응답 하나 만드는 데 stringify 52ms

→ 같은 요청 열 번 → 루프 0.5초 정지

→ 데이터가 쌓일수록 길어짐

→ 고치는 방향: 페이지로 나누기, 조금씩 만들어 흘려보내기

JSON.parse

요청 본문 `JSON.parse`: Express가 같은 스레드에서 실행, 본문 크기 기본 제한 100KB

제한을 풀어야 하면 그 API 하나만

데이터 블록: body-parser 기본 limit (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

정규식 /^(a+)+$/

입력	걸린 시간
a 20개 + !	33ms
a 24개 + !	69ms
a 26개 + !	291ms
a 28개 + !	911ms

→ 맞지 않는 입력에서 가능한 나누기를 전부 시도 (되돌아가기)

→ 글자 두 개마다 약 세 배

데이터 블록: 2026-10-26 강사 PC 실측 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

평소엔 안 보이는 이유

→ 같은 코드, 입력 크기만 다름

→ 확인할 곳: 요청마다 도는 코드 안의 Sync 함수, 큰 JSON, 사용자 입력에 거는 정규식

개발

개발 데이터: 수십 건, 짧은 입력

운영

운영: 쌓인 데이터, 사용자가 넣은 긴 입력

© 한상훈 NestJS 백엔드 마스터

평균 응답 시간에 안 보이는 멈춤

1분에 한 번

1분에 한 번 0.3초 멈춤 → 평균 응답 시간은 거의 그대로

그 순간

멈춘 순간의 손님만 느림

잴 것

재야 할 것: 루프가 늦게 돈 시간

© 한상훈 NestJS 백엔드 마스터

monitorEventLoopDelay

```
import { monitorEventLoopDelay } from 'node:perf_hooks';

const h = monitorEventLoopDelay({ resolution: 1 });
h.enable();

setInterval(() => {
  console.log(`p99 ${(h.percentile(99) / 1e6).toFixed(1)}ms`,
              `max ${(h.max / 1e6).toFixed(1)}ms`);
  h.reset();
}, 10_000);
```

→ 값의 단위: 나노초 → 1e6으로 나눠 밀리초

→ NestJS: `main.ts`에서 앱 생성 전에, 앱 전체가 루프 하나

© 한상훈 NestJS 백엔드 마스터

한가할 때와 막힐 때

상태	p50	p99	max
3초 동안 한가함	1.0ms	1.3ms	2.6ms
300ms 동기 작업 6번 섞임	1.0ms	1.7ms	303.0ms

→ 짧게 자주 재는 값이라 p99에도 안 잡히는 멈춤 → max 확인

→ 윈도우: 한가할 때도 약 15ms (타이머 해상도), 운영과 같은 리눅스에서 측정

데이터 블록: 2026-10-26 WSL2 Ubuntu 24.04, Node v22.22.0 실측 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

기다리는 일과 손이 묶이는 일

카페	Node 서버	루프
샷 걸어 두기	`await readFile`, DB 쿼리, `fetch`	계속 돎
원두 손으로 갈기	Sync 함수, 큰 JSON, 무거운 계산, 정규식	멈춤

© 한상훈 NestJS 백엔드 마스터

스레드 하나의 규칙

"이벤트 루프는 스레드 하나라서, 동기 코드 하나가 멈추면 모든 요청이 같이 멈춥니다."